

Excellent 👍 Let's go **step-by-step** through all **Servlet concepts** — from **basic** to **advanced**, including lifecycle, methods, request handling, and real project use.

What is a Servlet?

A **Servlet** is a Java program that runs on a **web server (like Tomcat)** and handles **requests and responses** between the **client (browser)** and **server-side logic**.

🧩 It is part of **Java EE (Jakarta EE)** — used to create **dynamic web applications**.

Servlet Architecture

SCSS

 Copy code

```
Client (Browser)
  ↓ HTTP Request
Web Server (Tomcat)
  ↓
Servlet Container (Engine)
  ↓
Servlet Program
  ↓
Response sent back to client
```

Key Features

- Platform-independent (Java-based)
 - Efficient and secure
 - Portable across servers
 - Handles multiple requests using threads
 - Extensible — integrates with JSP, JDBC, etc.
-



Servlet Lifecycle

Every servlet follows this **lifecycle (defined in `javax.servlet.Servlet` interface)**:

Step	Method	Description
1	<code>init()</code>	Called once when the servlet is created. Used for initialization.
2	<code>service()</code>	Called for each request. Decides which method (GET/POST) to call.
3	<code>doGet()</code> / <code>doPost()</code>	Handles HTTP GET/POST requests.
4	<code>destroy()</code>	Called once before servlet is removed from memory. Used for cleanup.



Example: Basic Servlet Lifecycle Code

java

Copy code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LifeCycleDemo extends HttpServlet {
    public void init() throws ServletException {
        System.out.println("Servlet initialized");
    }

    public void service(HttpServletRequest req, HttpServletResponse res)
        throws IOException, ServletException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h3>Service method executed!</h3>");
    }

    public void destroy() {
        System.out.println("Servlet destroyed");
    }
}
```

```
}  
}
```

HTTP Methods in Servlet

Method	Description
<code>doGet()</code>	Handles form data from the URL (GET method).
<code>doPost()</code>	Handles form data sent in the body (POST method).
<code>doPut()</code>	Updates resources.
<code>doDelete()</code>	Deletes resources.
<code>doHead()</code> , <code>doOptions()</code> , <code>doTrace()</code>	Other advanced methods.

Example: `doGet()` and `doPost()`

java

 Copy code

```
import java.io.*;  
import javax.servlet.*;  
import javax.servlet.http.*;  
  
public class LoginServlet extends HttpServlet {  
    protected void doGet(HttpServletRequest req, HttpServletResponse res)  
        throws ServletException, IOException {  
        res.setContentType("text/html");  
        PrintWriter out = res.getWriter();  
        out.println("<form action='LoginServlet' method='post'>");  
        out.println("Username: <input type='text' name='user'><br>");  
        out.println("Password: <input type='password' name='pass'><br>");  
        out.println("<input type='submit' value='Login'>");  
        out.println("</form>");  
    }  
  
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
```

```
        throws ServletException, IOException {  
    String user = req.getParameter("user");  
    String pass = req.getParameter("pass");  
  
    res.setContentType("text/html");  
    PrintWriter out = res.getWriter();  
    if (user.equals("admin") && pass.equals("1234")) {  
        out.println("<h2>Welcome " + user + "!</h2>");  
    } else {  
        out.println("<h3>Invalid login</h3>");  
    }  
    }  
}
```



Request & Response Objects

HttpServletRequest

Used to get data from client/browser.

java

 Copy code

```
String name = request.getParameter("username");  
String city = request.getParameter("city");
```

HttpServletResponse

Used to send response back to client.

java

 Copy code

```
response.setContentType("text/html");  
PrintWriter out = response.getWriter();  
out.println("<h1>Hello, " + name + "</h1>");
```



ServletConfig vs ServletContext

Feature	ServletConfig	ServletContext
Scope	Specific to one servlet	Shared by all servlets
Usage	Init parameters for one servlet	Global parameters for whole app
Example	DB config for single servlet	Common info like admin email

xml

 Copy code

```
<!-- web.xml -->
<servlet>
  <servlet-name>DBServlet</servlet-name>
  <servlet-class>DatabaseServlet</servlet-class>
  <init-param>
    <param-name>dbURL</param-name>
    <param-value>jdbc:mysql://localhost/test</param-value>
  </init-param>
</servlet>
```

Session Tracking in Servlets

Used to maintain user data between requests.

Methods:

1. Cookies
2. Hidden Form Fields
3. URL Rewriting
4. HttpSession

Example using HttpSession :

java

 Copy code

```
HttpSession session = request.getSession();
session.setAttribute("username", "Yuvraj");

String name = (String) session.getAttribute("username");
```

RequestDispatcher

Used to forward or include another servlet or JSP.

java

 Copy code

```
RequestDispatcher rd = request.getRequestDispatcher("home.jsp");  
rd.forward(request, response);
```

Or include:

java

 Copy code

```
rd.include(request, response);
```

Annotation-Based Servlet (No web.xml)

java

 Copy code

```
import javax.servlet.annotation.WebServlet;  
  
@WebServlet("/hello")  
public class HelloServlet extends HttpServlet {  
    protected void doGet(HttpServletRequest req, HttpServletResponse res)  
        throws IOException {  
        res.setContentType("text/html");  
        res.getWriter().println("<h1>Hello from Annotation Servlet!</h1>");  
    }  
}
```

Servlet + JDBC Integration Example

java

 Copy code

```
import java.sql.*;  
import javax.servlet.*;  
import javax.servlet.http.*;
```

```
import java.io.*;

@WebServlet("/login")
public class LoginDBServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        String user = req.getParameter("username");
        String pass = req.getParameter("password");

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "root");

            PreparedStatement ps = con.prepareStatement(
                "SELECT * FROM users WHERE username=? AND password=?");
            ps.setString(1, user);
            ps.setString(2, pass);

            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                out.println("<h2>Welcome, " + user + "</h2>");
            } else {
                out.println("<h3>Invalid username or password</h3>");
            }

            con.close();
        } catch (Exception e) {
            e.printStackTrace(out);
        }
    }
}
```



Advanced Concepts

- **Filters:** Modify request/response before reaching servlet.

- **Listeners:** Track changes in session, context, or attributes.
- **Asynchronous Servlets:** Handle long-running requests.
- **File Upload / Download** using `MultipartConfig`.
- **Error Handling** using `web.xml` or annotations.

Typical Project Structure

pgsql

 Copy code

WebApp/

```
|— WEB-INF/
|   |— web.xml
|   |— lib/
|   |— classes/
|— index.html
|— login.html
|— WEB-INF/classes/LoginServlet.class
```

Would you like me to prepare a **mini project using Servlet + JDBC (like a Student Management or Login System)** next?

That will include **frontend form + servlet + database integration** (complete code).

   ...

Yee

Great! 🔥

Let's build a **complete Servlet + JDBC Mini Project** — clean, simple, and perfect for interviews and college practicals.

You will get:

- ✓ Full project structure
- ✓ web.xml
- ✓ HTML pages
- ✓ Servlet classes
- ✓ JDBC connection
- ✓ CRUD operations
- ✓ Step-by-step explanation

MINI PROJECT 1: Student Management System using Servlet + JDBC

(INSERT, VIEW, UPDATE, DELETE)

✓ 1. Project Structure (Eclipse Dynamic Web Project)

pgsql

 Copy code

```
StudentManagement/
├── src/
│   ├── com.project.dao/
│   │   └── StudentDAO.java
│   ├── com.project.model/
│   │   └── Student.java
│   └── com.project.servlet/
│       ├── AddStudentServlet.java
│       ├── ViewStudentServlet.java
│       ├── UpdateStudentServlet.java
│       └── DeleteStudentServlet.java
└── WebContent/
    ├── index.html
    ├── add.html
    ├── update.html
    ├── view.jsp
    └── WEB-INF/
        └── web.xml
```

```
└─ lib/
    └─ mysql-connector.jar
```

✓ 2. Database Setup (MySQL)

sql

 Copy code

```
CREATE DATABASE studentdb;

USE studentdb;

CREATE TABLE students (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    email VARCHAR(50),
    city VARCHAR(50)
);
```

✓ 3. Student Model (POJO)

 File: *Student.java*

java

 Copy code

```
package com.project.model;

public class Student {
    private int id;
    private String name;
    private String email;
    private String city;

    public Student() {}

    public Student(int id, String name, String email, String city) {
        this.id = id;
        this.name = name;
    }
}
```

```
        this.email = email;
        this.city = city;
    }

    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getCity() { return city; }
    public void setCity(String city) { this.city = city; }
}
```

4. DAO Class for Database Operations

 File: StudentDAO.java

java

 Copy code

```
package com.project.dao;

import java.sql.*;
import com.project.model.Student;

public class StudentDAO {
    static Connection con;

    public static Connection getConnection() {
        try {
            if (con == null) {
                Class.forName("com.mysql.cj.jdbc.Driver");
                con = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/studentdb", "root", "root"
                );
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return con;
    }
}
```

```
    }  
    } catch (Exception e) { e.printStackTrace(); }  
    return con;  
}  
  
public static int addStudent(Student s) {  
    int status = 0;  
    try {  
        PreparedStatement ps = getConnection().prepareStatement(  
            "INSERT INTO students(name,email,city) VALUES(?,?,?)"  
        );  
        ps.setString(1, s.getName());  
        ps.setString(2, s.getEmail());  
        ps.setString(3, s.getCity());  
        status = ps.executeUpdate();  
    } catch (Exception e) { e.printStackTrace(); }  
    return status;  
}  
  
public static ResultSet getStudents() {  
    ResultSet rs = null;  
    try {  
        PreparedStatement ps = getConnection().prepareStatement(  
            "SELECT * FROM students"  
        );  
        rs = ps.executeQuery();  
    } catch (Exception e) { e.printStackTrace(); }  
    return rs;  
}  
  
public static int updateStudent(Student s) {  
    int status = 0;  
    try {  
        PreparedStatement ps = getConnection().prepareStatement(  
            "UPDATE students SET name=?, email=?, city=? WHERE id=?"  
        );  
        ps.setString(1, s.getName());  
        ps.setString(2, s.getEmail());  
        ps.setString(3, s.getCity());  
        ps.setInt(4, s.getId());  
    }
```

```
        status = ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); }
    return status;
}

public static int deleteStudent(int id) {
    int status = 0;
    try {
        PreparedStatement ps = getConnection().prepareStatement(
            "DELETE FROM students WHERE id=?"
        );
        ps.setInt(1, id);
        status = ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); }
    return status;
}
}
```

✓ 5. Add Student Servlet

 File: *AddStudentServlet.java*

java

 Copy code

```
package com.project.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.project.dao.StudentDAO;
import com.project.model.Student;

public class AddStudentServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        String name = req.getParameter("name");
        String email = req.getParameter("email");
        String city = req.getParameter("city");
    }
}
```

```
Student s = new Student(0, name, email, city);

int status = StudentDAO.addStudent(s);

res.setContentType("text/html");
PrintWriter out = res.getWriter();

if (status > 0)
    out.println("<h3>Student Added Successfully</h3>");
else
    out.println("<h3>Error adding student</h3>");
}
```

✓ 6. View Students Servlet

 File: *ViewStudentServlet.java*

java

 Copy code

```
package com.project.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import java.sql.*;
import com.project.dao.StudentDAO;

public class ViewStudentServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        out.println("<h2>Student List</h2>");
        out.println("<table border='1'><tr><th>ID</th><th>Name</th><th>Email</th><th>City</th></tr></table>");
    }
}
```

```
try {
    ResultSet rs = StudentDAO.getStudents();
    while (rs.next()) {
        out.println("<tr><td>" + rs.getInt(1) + "</td><td>" +
            rs.getString(2) + "</td><td>" +
            rs.getString(3) + "</td><td>" +
            rs.getString(4) + "</td></tr>");
    }
} catch (Exception e) {
    out.println("<h3>Error: " + e + "</h3>");
}

out.println("</table>");
}
```

✓ 7. Update Student Servlet

 File: *UpdateStudentServlet.java*

java

 Copy code

```
package com.project.servlet;

import com.project.dao.StudentDAO;
import com.project.model.Student;
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class UpdateStudentServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));
        String name = req.getParameter("name");
        String email = req.getParameter("email");
        String city = req.getParameter("city");
    }
}
```

```
Student s = new Student(id, name, email, city);

int status = StudentDAO.updateStudent(s);

res.setContentType("text/html");
PrintWriter out = res.getWriter();

if (status > 0)
    out.println("<h3>Student Updated Successfully</h3>");
else
    out.println("<h3>Error updating student</h3>");
}
```

✓ 8. Delete Student Servlet

java

 Copy code

```
package com.project.servlet;

import com.project.dao.StudentDAO;
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class DeleteStudentServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));

        int status = StudentDAO.deleteStudent(id);

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        if (status > 0)
            out.println("<h3>Student Deleted Successfully</h3>");
    }
}
```

```
        else
            out.println("<h3>Error deleting student</h3>");
    }
}
```

9. web.xml Configuration

xml

 Copy code

```
<web-app>

    <servlet>
        <servlet-name>add</servlet-name>
        <servlet-class>com.project.servlet.AddStudentServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>add</servlet-name>
        <url-pattern>/add</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>view</servlet-name>
        <servlet-class>com.project.servlet.ViewStudentServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>view</servlet-name>
        <url-pattern>/view</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>update</servlet-name>
        <servlet-class>com.project.servlet.UpdateStudentServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>update</servlet-name>
        <url-pattern>/update</url-pattern>
    </servlet-mapping>

    <servlet>
```

```
<servlet-name>delete</servlet-name>
<servlet-class>com.project.servlet.DeleteStudentServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>delete</servlet-name>
  <url-pattern>/delete</url-pattern>
</servlet-mapping>

</web-app>
```



10. HTML Pages

► index.html

html

 Copy code

```
<h2>Student Management</h2>
<a href="add.html">Add Student</a><br>
<a href="update.html">Update Student</a><br>
<a href="delete.html">Delete Student</a><br>
<a href="view">View Students</a><br>
```

► add.html

html

 Copy code

```
<form action="add" method="post">
  Name: <input type="text" name="name"><br>
  Email: <input type="text" name="email"><br>
  City: <input type="text" name="city"><br>
  <input type="submit" value="Add Student">
</form>
```



PROJECT IS COMPLETE!

If you want:

✓ Login + Register Servlet Project

OR

✓ Employee CRUD Project

OR

✓ Product Inventory Servlet Project

Just tell me “Next project” and which one you want.



Next

Great! 🔥

Next, I will give you a complete **Servlet + JDBC Login & Registration System** — simple, clean, and perfect for college projects + interviews.

You will get:

- ✓ Login + Register
- ✓ Validation
- ✓ JDBC
- ✓ Servlet classes
- ✓ web.xml
- ✓ HTML pages
- ✓ MySQL table
- ✓ Session handling

🎓 **MINI PROJECT 2: Login & Registration System (Servlet + JDBC)**

✓ **1. Project Structure**

pgsql

```
LoginSystem/
├── src/
│   ├── com.project.dao/
│   │   └── UserDAO.java
│   ├── com.project.model/
│   │   └── User.java
│   └── com.project.servlet/
│       ├── RegisterServlet.java
│       ├── LoginServlet.java
│       └── LogoutServlet.java
├── WebContent/
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   ├── home.jsp
│   └── WEB-INF/
│       └── web.xml
└── lib/
    └── mysql-connector.jar
```

✓ 2. MySQL Database

sql

 Copy code

```
CREATE DATABASE logindb;
```

```
USE logindb;
```

```
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    password VARCHAR(200)
);
```

✓ 3. User Model (POJO)

 File: *User.java*

java

 Copy code

```
package com.project.model;

public class User {
    private int id;
    private String name;
    private String email;
    private String password;

    public User() {}

    public User(String name, String email, String password) {
        this.name = name;
        this.email = email;
        this.password = password;
    }

    // Getters & Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

✓ 4. DAO for Database Operations

 File: *UserDAO.java*

java

 Copy code

```
package com.project.dao;

import java.sql.*;
import com.project.model.User;

public class UserDao {
    static Connection con;

    public static Connection getConnection() {
        try {
            if (con == null) {
                Class.forName("com.mysql.cj.jdbc.Driver");
                con = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/login_db", "root", "root"
                );
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return con;
    }

    public static int register(User u) {
        int status = 0;
        try {
            PreparedStatement ps = getConnection().prepareStatement(
                "INSERT INTO users(name,email,password) VALUES(?,?,?)"
            );
            ps.setString(1, u.getName());
            ps.setString(2, u.getEmail());
            ps.setString(3, u.getPassword());
            status = ps.executeUpdate();
        } catch (Exception e) { e.printStackTrace(); }
        return status;
    }

    public static User login(String email, String password) {
        User u = null;
```

```
try {
    PreparedStatement ps = getConnection().prepareStatement(
        "SELECT * FROM users WHERE email=? AND password=?"
    );
    ps.setString(1, email);
    ps.setString(2, password);
    ResultSet rs = ps.executeQuery();

    if (rs.next()) {
        u = new User();
        u.setId(rs.getInt("id"));
        u.setName(rs.getString("name"));
        u.setEmail(rs.getString("email"));
    }
} catch (Exception e) { e.printStackTrace(); }
return u;
}
```

✓ 5. Register Servlet

 File: RegisterServlet.java

java

 Copy code

```
package com.project.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.project.dao.UserDAO;
import com.project.model.User;

public class RegisterServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        String name = req.getParameter("name");
        String email = req.getParameter("email");
```

```
String pass = req.getParameter("password");

User u = new User(name, email, pass);
int status = UserDAO.register(u);

res.setContentType("text/html");
PrintWriter out = res.getWriter();

if (status > 0)
    out.println("<h3>Registration Successful</h3><a href='login.html'>Log
else
    out.println("<h3>Error during registration</h3>");
}
```

✓ 6. Login Servlet

 File: *LoginServlet.java*

java

 Copy code

```
package com.project.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.project.dao.UserDAO;
import com.project.model.User;

public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        String email = req.getParameter("email");
        String password = req.getParameter("password");

        User u = UserDAO.login(email, password);
```

```
        if (u != null) {
            HttpSession session = req.getSession();
            session.setAttribute("user", u);

            res.sendRedirect("home.jsp");
        } else {
            res.setContentType("text/html");
            PrintWriter out = res.getWriter();
            out.println("<h3>Invalid Credentials</h3>");
        }
    }
}
```

✓ 7. Logout Servlet

 File: LogoutServlet.java

java

 Copy code

```
package com.project.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        HttpSession session = req.getSession();
        session.invalidate();

        res.sendRedirect("login.html");
    }
}
```

✓ 8. web.xml Configuration

xml

 Copy code

```
<web-app>

  <servlet>
    <servlet-name>register</servlet-name>
    <servlet-class>com.project.servlet.RegisterServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>register</servlet-name>
    <url-pattern>/register</url-pattern>
  </servlet-mapping>

  <servlet>
    <servlet-name>login</servlet-name>
    <servlet-class>com.project.servlet.LoginServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>login</servlet-name>
    <url-pattern>/login</url-pattern>
  </servlet-mapping>

  <servlet>
    <servlet-name>logout</servlet-name>
    <servlet-class>com.project.servlet.LogoutServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>logout</servlet-name>
    <url-pattern>/logout</url-pattern>
  </servlet-mapping>

</web-app>
```



9. HTML Pages

► index.html

html

 Copy code

```
<h2>Welcome</h2>
<a href="login.html">Login</a><br>
<a href="register.html">Register</a>
```

► register.html

html

 Copy code

```
<h2>Register</h2>
<form action="register" method="post">
  Name: <input type="text" name="name"><br>
  Email: <input type="text" name="email"><br>
  Password: <input type="password" name="password"><br>
  <input type="submit" value="Register">
</form>
```

► login.html

html

 Copy code

```
<h2>Login</h2>
<form action="login" method="post">
  Email: <input type="text" name="email"><br>
  Password: <input type="password" name="password"><br>
  <input type="submit" value="Login">
</form>
```

home.jsp (After Login)

jsp

 Copy code

```
<%@ page import="com.project.model.User" %>

<%
  User u = (User) session.getAttribute("user");

  if (u == null) {
    response.sendRedirect("login.html");
  }
%>
```

```
}  
%>  
  
<h2>Welcome, <%= u.getName() %></h2>  
  
<a href="logout">Logout</a>
```



PROJECT COMPLETE!

If you want next project:

- 1 Employee CRUD
- 2 Product Inventory System
- 3 Online Examination Servlet Project
- 4 Hotel Booking Servlet Project

Just reply "Next + project number".



1

Great! 🚀

Let's start **Servlet + JDBC Mini Project 1: Employee Management System (CRUD)**.

This is one of the best and most commonly asked projects in **viva, exams, and interviews**.

You will get:

- ✓ Add Employee
- ✓ View Employees
- ✓ Update Employee
- ✓ Delete Employee
- ✓ JDBC + DAO
- ✓ Servlets + JSP

- ✓ MySQL
- ✓ Complete Project Structure



PROJECT: Employee Management System (CRUD)

(Perfect for Eclipse + Tomcat)



1. Database Setup

Run this in MySQL:

sql

Copy code

```
CREATE DATABASE employeedb;
```

```
USE employeedb;
```

```
CREATE TABLE employee (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    salary DOUBLE  
);
```



2. Project Structure

pgsql

Copy code

```
EmployeeCRUD/  
├── src/  
│   ├── com.employee.model/      -> Employee.java  
│   ├── com.employee.dao/        -> EmployeeDAO.java  
│   └── com.employee.servlet/    -> Add, View, Edit, Update, Delete Servlets  
├── WebContent/  
│   ├── add.jsp  
│   └── view.jsp
```

```
|      |— edit.jsp
|      |— index.jsp
|      └ WEB-INF/web.xml
└ lib/mysql-connector.jar
```

✓ 3. Employee Model (POJO)

✦ *File: Employee.java*

java

Copy code

```
package com.employee.model;

public class Employee {
    private int id;
    private String name;
    private String email;
    private double salary;

    public Employee() {}

    public Employee(String name, String email, double salary) {
        this.name = name;
        this.email = email;
        this.salary = salary;
    }

    public Employee(int id, String name, String email, double salary) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.salary = salary;
    }

    // Getters & Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
```

```
public void setName(String name) { this.name = name; }

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public double getSalary() { return salary; }
public void setSalary(double salary) { this.salary = salary; }
}
```



4. Employee DAO (Database Operations)

✦ File: EmployeeDAO.java

java

Copy code

```
package com.employee.dao;

import java.sql.*;
import java.util.*;
import com.employee.model.Employee;

public class EmployeeDAO {

    private static Connection getConnection() {
        Connection con = null;
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/employeeedb", "root", "root"
            );
        } catch (Exception e) { e.printStackTrace(); }
        return con;
    }

    public static int addEmployee(Employee e) {
        int status = 0;
        try {
            Connection con = getConnection();
            PreparedStatement ps = con.prepareStatement(
```

```
        "INSERT INTO employee(name,email,salary) VALUES(?,?,?)"
    );
    ps.setString(1, e.getName());
    ps.setString(2, e.getEmail());
    ps.setDouble(3, e.getSalary());

    status = ps.executeUpdate();
} catch (Exception ex) { ex.printStackTrace(); }
return status;
}

public static List<Employee> getAllEmployees() {
    List<Employee> list = new ArrayList<>();
    try {
        Connection con = getConnection();
        PreparedStatement ps = con.prepareStatement("SELECT * FROM employee")
        ResultSet rs = ps.executeQuery();

        while (rs.next()) {
            list.add(new Employee(
                rs.getInt("id"),
                rs.getString("name"),
                rs.getString("email"),
                rs.getDouble("salary")
            ));
        }
    } catch (Exception e) { e.printStackTrace(); }
    return list;
}

public static Employee getEmployeeById(int id) {
    Employee e = null;
    try {
        Connection con = getConnection();
        PreparedStatement ps = con.prepareStatement("SELECT * FROM employee WHERE id=?")
        ps.setInt(1, id);

        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            e = new Employee(
```

```
        rs.getInt("id"),
        rs.getString("name"),
        rs.getString("email"),
        rs.getDouble("salary")
    );
}
} catch (Exception ex) { ex.printStackTrace(); }
return e;
}

public static int updateEmployee(Employee e) {
    int status = 0;
    try {
        Connection con = getConnection();
        PreparedStatement ps = con.prepareStatement(
            "UPDATE employee SET name=?, email=?, salary=? WHERE id=?"
        );
        ps.setString(1, e.getName());
        ps.setString(2, e.getEmail());
        ps.setDouble(3, e.getSalary());
        ps.setInt(4, e.getId());

        status = ps.executeUpdate();
    } catch (Exception ex) { ex.printStackTrace(); }
    return status;
}

public static int deleteEmployee(int id) {
    int status = 0;
    try {
        Connection con = getConnection();
        PreparedStatement ps = con.prepareStatement("DELETE FROM employee WHERE id=?");
        ps.setInt(1, id);

        status = ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); }
    return status;
}
}
```

✓ 5. Add Employee Servlet

✚ File: *AddEmployeeServlet.java*

java

📋 Copy code

```
package com.employee.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.employee.model.Employee;
import com.employee.dao.EmployeeDAO;

public class AddEmployeeServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        String name = req.getParameter("name");
        String email = req.getParameter("email");
        double salary = Double.parseDouble(req.getParameter("salary"));

        Employee e = new Employee(name, email, salary);
        int status = EmployeeDAO.addEmployee(e);

        if (status > 0) {
            res.sendRedirect("view");
        } else {
            res.getWriter().println("Error adding employee!");
        }
    }
}
```

✓ 6. View Employees Servlet

✚ File: *ViewServlet.java*

java

```
package com.employee.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import java.util.*;
import com.employee.dao.EmployeeDAO;
import com.employee.model.Employee;

public class ViewServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        List<Employee> list = EmployeeDAO.getAllEmployees();
        req.setAttribute("list", list);
        RequestDispatcher rd = req.getRequestDispatcher("view.jsp");
        rd.forward(req, res);
    }
}
```

✓ 7. Edit Servlet (Load Employee Data)

✦ File: EditServlet.java

java

 Copy code

```
package com.employee.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.employee.dao.EmployeeDAO;
import com.employee.model.Employee;

public class EditServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
```

```
int id = Integer.parseInt(req.getParameter("id"));
Employee e = EmployeeDAO.getEmployeeById(id);

req.setAttribute("emp", e);
RequestDispatcher rd = req.getRequestDispatcher("edit.jsp");
rd.forward(req, res);
}
}
```

✓ 8. Update Servlet

✦ File: *UpdateServlet.java*

java

📋 Copy code

```
package com.employee.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.employee.model.Employee;
import com.employee.dao.EmployeeDAO;

public class UpdateServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));
        String name = req.getParameter("name");
        String email = req.getParameter("email");
        double salary = Double.parseDouble(req.getParameter("salary"));

        Employee e = new Employee(id, name, email, salary);
        int status = EmployeeDAO.updateEmployee(e);

        if (status > 0)
            res.sendRedirect("view");
        else
            res.getWriter().println("Error updating employee!");
    }
}
```

```
}  
}
```

✓ 9. Delete Servlet

✦ File: DeleteServlet.java

java

Copy code

```
package com.employee.servlet;  
  
import javax.servlet.*;  
import javax.servlet.http.*;  
import java.io.*;  
import com.employee.dao.EmployeeDAO;  
  
public class DeleteServlet extends HttpServlet {  
    protected void doGet(HttpServletRequest req, HttpServletResponse res)  
        throws ServletException, IOException {  
  
        int id = Integer.parseInt(req.getParameter("id"));  
        EmployeeDAO.deleteEmployee(id);  
  
        res.sendRedirect("view");  
    }  
}
```

✓ 10. web.xml Mapping

xml

Copy code

```
<web-app>  
  
    <servlet>  
        <servlet-name>add</servlet-name>  
        <servlet-class>com.employee.servlet.AddEmployeeServlet</servlet-class>  
    </servlet>
```

```
<servlet-mapping>
  <servlet-name>add</servlet-name>
  <url-pattern>/add</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>view</servlet-name>
  <servlet-class>com.employee.servlet.ViewServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>view</servlet-name>
  <url-pattern>/view</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>edit</servlet-name>
  <servlet-class>com.employee.servlet.EditServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>edit</servlet-name>
  <url-pattern>/edit</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>update</servlet-name>
  <servlet-class>com.employee.servlet.UpdateServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>update</servlet-name>
  <url-pattern>/update</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>delete</servlet-name>
  <servlet-class>com.employee.servlet.DeleteServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>delete</servlet-name>
  <url-pattern>/delete</url-pattern>
</servlet-mapping>
```

</web-app>

✓ 11. JSP Pages

✚ index.jsp

jsp

📋 Copy code

```
<h2>Employee Management System</h2>
<a href="add.jsp">Add Employee</a> |
<a href="view">View Employees</a>
```

✚ add.jsp

jsp

📋 Copy code

```
<h2>Add New Employee</h2>

<form action="add" method="post">
  Name: <input type="text" name="name"><br>
  Email: <input type="text" name="email"><br>
  Salary: <input type="text" name="salary"><br>
  <input type="submit" value="Add">
</form>
```

✚ view.jsp

jsp

📋 Copy code

```
<%@ page import="java.util.*, com.employee.model.Employee" %>

<h2>Employee List</h2>
```

```
<table border="1">
<tr>
  <th>ID</th>
  <th>Name</th>
  <th>Email</th>
  <th>Salary</th>
  <th>Actions</th>
</tr>

<%
  List<Employee> list = (List<Employee>) request.getAttribute("list");
  for (Employee e : list) {
%>
<tr>
  <td><%= e.getId() %></td>
  <td><%= e.getName() %></td>
  <td><%= e.getEmail() %></td>
  <td><%= e.getSalary() %></td>
  <td>
    <a href="edit?id=<%= e.getId() %>">Edit</a> |
    <a href="delete?id=<%= e.getId() %>">Delete</a>
  </td>
</tr>
<% } %>

</table>
```

edit.jsp

jsp

 Copy code

```
<%@ page import="com.employee.model.Employee" %>

<%
  Employee e = (Employee) request.getAttribute("emp");
%>

<h2>Edit Employee</h2>
```

```
<form action="update" method="post">
  <input type="hidden" name="id" value="<%= e.getId() %>">

  Name: <input type="text" name="name" value="<%= e.getName() %>"><br>
  Email: <input type="text" name="email" value="<%= e.getEmail() %>"><br>
  Salary: <input type="text" name="salary" value="<%= e.getSalary() %>"><br>
  <input type="submit" value="Update">
</form>
```



EMPLOYEE CRUD PROJECT COMPLETED

If you want next:

- 2 Product Inventory System
- 3 Hospital/Hotel Booking System
- 4 Student Management System
- 5 Online Exam System

Just reply with **project number**.



2

Great! 🚀

Here is your **Project 2: Product Inventory Management System (Servlet + JDBC)**

This is a **full CRUD project** — Add, View, Update, Delete products.

Perfect for mini-project, viva, Java interview, and servlet practice.



PRODUCT INVENTORY SYSTEM (SERVLET + JDBC)

You will get:

- ✓ Add product
- ✓ List all products
- ✓ Edit product
- ✓ Delete product
- ✓ JDBC DAO
- ✓ Servlet classes
- ✓ JSP pages
- ✓ MySQL table
- ✓ MVC structure

1. Project Folder Structure

bash

 Copy code

```
ProductInventory/  
├── src/  
│   ├── com.inventory.model/  
│   │   └── Product.java  
│   ├── com.inventory.dao/  
│   │   └── ProductDAO.java  
│   └── com.inventory.servlet/  
│       ├── AddProductServlet.java  
│       ├── EditProductServlet.java  
│       ├── UpdateProductServlet.java  
│       └── DeleteProductServlet.java  
├── WebContent/  
│   ├── add_product.jsp  
│   ├── list_products.jsp  
│   ├── edit_product.jsp  
│   ├── index.html  
│   └── WEB-INF/web.xml  
└── lib/mysql-connector.jar
```

2. MySQL Table

sql

```
CREATE DATABASE inventorydb;

USE inventorydb;

CREATE TABLE products (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    category VARCHAR(100),
    price DOUBLE,
    quantity INT
);
```



3. Product Model (POJO)

✦ *File: Product.java*

java

 Copy code

```
package com.inventory.model;

public class Product {
    private int id;
    private String name;
    private String category;
    private double price;
    private int quantity;

    public Product() {}

    public Product(String name, String category, double price, int quantity) {
        this.name = name;
        this.category = category;
        this.price = price;
        this.quantity = quantity;
    }

    // Getters & Setters
    public int getId() { return id; }
```

```
public void setId(int id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getCategory() { return category; }
public void setCategory(String category) { this.category = category; }

public double getPrice() { return price; }
public void setPrice(double price) { this.price = price; }

public int getQuantity() { return quantity; }
public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

4. Product DAO (JDBC Logic)

✦ *File: ProductDAO.java*

java

 Copy code

```
package com.inventory.dao;

import java.sql.*;
import java.util.*;
import com.inventory.model.Product;

public class ProductDAO {
    static Connection con;

    public static Connection getConnection() {
        try {
            if (con == null) {
                Class.forName("com.mysql.cj.jdbc.Driver");
                con = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/inventorydb", "root", "root"
                );
            }
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

```
        return con;
    }

    // INSERT
    public static int addProduct(Product p) {
        int status = 0;
        try {
            PreparedStatement ps = getConnection().prepareStatement(
                "INSERT INTO products(name, category, price, quantity) VALUES(?,?)");
            ps.setString(1, p.getName());
            ps.setString(2, p.getCategory());
            ps.setDouble(3, p.getPrice());
            ps.setInt(4, p.getQuantity());

            status = ps.executeUpdate();
        } catch (Exception e) { e.printStackTrace(); }
        return status;
    }

    // SELECT ALL
    public static List<Product> getAllProducts() {
        List<Product> list = new ArrayList<>();
        try {
            PreparedStatement ps = getConnection().prepareStatement(
                "SELECT * FROM products");
            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                Product p = new Product();
                p.setId(rs.getInt("id"));
                p.setName(rs.getString("name"));
                p.setCategory(rs.getString("category"));
                p.setPrice(rs.getDouble("price"));
                p.setQuantity(rs.getInt("quantity"));
                list.add(p);
            }

        } catch (Exception e) { e.printStackTrace(); }
```

```
        return list;
    }

    // SELECT ONE
    public static Product getProductById(int id) {
        Product p = null;
        try {
            PreparedStatement ps = getConnection().prepareStatement(
                "SELECT * FROM products WHERE id=?"
            );
            ps.setInt(1, id);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                p = new Product();
                p.setId(rs.getInt("id"));
                p.setName(rs.getString("name"));
                p.setCategory(rs.getString("category"));
                p.setPrice(rs.getDouble("price"));
                p.setQuantity(rs.getInt("quantity"));
            }

        } catch (Exception e) { e.printStackTrace(); }
        return p;
    }

    // UPDATE
    public static int updateProduct(Product p) {
        int status = 0;
        try {
            PreparedStatement ps = getConnection().prepareStatement(
                "UPDATE products SET name=?, category=?, price=?, quantity=? WHERE id=?"
            );

            ps.setString(1, p.getName());
            ps.setString(2, p.getCategory());
            ps.setDouble(3, p.getPrice());
            ps.setInt(4, p.getQuantity());
            ps.setInt(5, p.getId());
        }
    }
}
```

```
        status = ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); }
    return status;
}

// DELETE
public static int deleteProduct(int id) {
    int status = 0;
    try {
        PreparedStatement ps = getConnection().prepareStatement(
            "DELETE FROM products WHERE id=?"
        );
        ps.setInt(1, id);
        status = ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); }
    return status;
}
}
```

5. Add Product Servlet

 *AddProductServlet.java*

java

 Copy code

```
package com.inventory.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.inventory.dao.ProductDAO;
import com.inventory.model.Product;

public class AddProductServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        String name = req.getParameter("name");
```

```
String category = req.getParameter("category");
double price = Double.parseDouble(req.getParameter("price"));
int quantity = Integer.parseInt(req.getParameter("quantity"));

Product p = new Product(name, category, price, quantity);
ProductDAO.addProduct(p);

res.sendRedirect("list_products.jsp");
}
}
```

6. Edit (Load Data) Servlet

 *EditProductServlet.java*

java

 Copy code

```
package com.inventory.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.inventory.dao.ProductDAO;
import com.inventory.model.Product;

public class EditProductServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));
        Product p = ProductDAO.getProductById(id);

        req.setAttribute("product", p);
        RequestDispatcher rd = req.getRequestDispatcher("edit_product.jsp");
        rd.forward(req, res);
    }
}
```

7. Update Product Servlet

 *UpdateProductServlet.java*

java

 Copy code

```
package com.inventory.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.inventory.dao.ProductDAO;
import com.inventory.model.Product;

public class UpdateProductServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));
        String name = req.getParameter("name");
        String category = req.getParameter("category");
        double price = Double.parseDouble(req.getParameter("price"));
        int quantity = Integer.parseInt(req.getParameter("quantity"));

        Product p = new Product(name, category, price, quantity);
        p.setId(id);

        ProductDAO.updateProduct(p);

        res.sendRedirect("list_products.jsp");
    }
}
```

8. Delete Servlet

 *DeleteProductServlet.java*

java

 Copy code

```
package com.inventory.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.inventory.dao.ProductDAO;

public class DeleteProductServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));
        ProductDAO.deleteProduct(id);

        res.sendRedirect("list_products.jsp");
    }
}
```

9. web.xml Configuration

xml

 Copy code

```
<web-app>

    <servlet>
        <servlet-name>addProduct</servlet-name>
        <servlet-class>com.inventory.servlet.AddProductServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>addProduct</servlet-name>
        <url-pattern>/addProduct</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>editProduct</servlet-name>
        <servlet-class>com.inventory.servlet.EditProductServlet</servlet-class>
    </servlet>
    <servlet-mapping>
```

```
<servlet-name>editProduct</servlet-name>
<url-pattern>/editProduct</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>updateProduct</servlet-name>
  <servlet-class>com.inventory.servlet.UpdateProductServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>updateProduct</servlet-name>
  <url-pattern>/updateProduct</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>deleteProduct</servlet-name>
  <servlet-class>com.inventory.servlet.DeleteProductServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>deleteProduct</servlet-name>
  <url-pattern>/deleteProduct</url-pattern>
</servlet-mapping>

</web-app>
```



10. JSP PAGES



index.html

html

Copy code

```
<h2>Product Inventory System</h2>
<a href="list_products.jsp">View Products</a><br>
<a href="add_product.jsp">Add Product</a>
```



add_product.jsp

jsp

 Copy code

```
<h2>Add Product</h2>
<form action="addProduct" method="post">
    Name: <input type="text" name="name"><br>
    Category: <input type="text" name="category"><br>
    Price: <input type="number" step="0.01" name="price"><br>
    Quantity: <input type="number" name="quantity"><br>
    <input type="submit" value="Add">
</form>
```

list_products.jsp

jsp

 Copy code

```
<%@ page import="java.util.*, com.inventory.dao.ProductDAO, com.inventory.model.P

<h2>All Products</h2>

<a href="add_product.jsp">Add New Product</a><br><br>

<table border="1" cellpadding="10">
<tr>
    <th>ID</th>
    <th>Name</th>
    <th>Category</th>
    <th>Price</th>
    <th>Qty</th>
    <th>Actions</th>
</tr>

<%
    List<Product> list = ProductDAO.getAllProducts();
    for(Product p : list) {
%>

<tr>
    <td><%= p.getId() %></td>
    <td><%= p.getName() %></td>
```

```

<td><%= p.getCategory() %></td>
<td><%= p.getPrice() %></td>
<td><%= p.getQuantity() %></td>
<td>
    <a href="editProduct?id=<%= p.getId() %>">Edit</a> |
    <a href="deleteProduct?id=<%= p.getId() %>">Delete</a>
</td>
</tr>

<% } %>
</table>

```

edit_product.jsp

jsp

 Copy code

```

<%@ page import="com.inventory.model.Product" %>

<%
    Product p = (Product) request.getAttribute("product");
%>

<h2>Edit Product</h2>

<form action="updateProduct" method="post">
    <input type="hidden" name="id" value="<%= p.getId() %>">
    Name: <input type="text" name="name" value="<%= p.getName() %>"><br>
    Category: <input type="text" name="category" value="<%= p.getCategory() %>"><br>
    Price: <input type="number" step="0.01" name="price" value="<%= p.getPrice() %>"><br>
    Quantity: <input type="number" name="quantity" value="<%= p.getQuantity() %>"><br>
    <input type="submit" value="Update">
</form>

```

 **PROJECT 100% COMPLETE!**

If you want:

- 1 Online Examination Servlet Project
- 2 Employee Leave Management Servlet Project
- 3 Library Management System (Servlet+JDBC)
- 4 Hotel Room Booking System

Just send "Next + project number".



1

Great! Let's start **Project 1: Student Management System (Servlet + JDBC)**

This is a complete **CRUD mini project** perfect for university submissions, viva, and servlet practice.

STUDENT MANAGEMENT SYSTEM (JAVA SERVLET + JDBC)

Features included:

- ✓ Add Student
- ✓ View All Students
- ✓ Update Student
- ✓ Delete Student
- ✓ MVC Architecture
- ✓ JDBC DAO
- ✓ JSP Frontend
- ✓ MySQL Database

1. Project Folder Structure

```
bash
```

```
StudentManagement/
├── src/
│   ├── com.student.model/Student.java
│   ├── com.student.dao/StudentDAO.java
│   └── com.student.servlet/
│       ├── AddStudentServlet.java
│       ├── EditStudentServlet.java
│       ├── UpdateStudentServlet.java
│       └── DeleteStudentServlet.java
├── WebContent/
│   ├── add_student.jsp
│   ├── list_students.jsp
│   ├── edit_student.jsp
│   ├── index.html
│   └── WEB-INF/web.xml
└── lib/mysql-connector.jar
```



2. MySQL Database

sql

Copy code

```
CREATE DATABASE studentdb;
```

```
USE studentdb;
```

```
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    course VARCHAR(100),
    fees DOUBLE
);
```



3. Student Model (POJO) – Student.java

java

```
package com.student.model;

public class Student {
    private int id;
    private String name;
    private String email;
    private String course;
    private double fees;

    public Student() {}

    public Student(String name, String email, String course, double fees) {
        this.name = name;
        this.email = email;
        this.course = course;
        this.fees = fees;
    }

    // Getters and setters

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getCourse() { return course; }
    public void setCourse(String course) { this.course = course; }

    public double getFees() { return fees; }
    public void setFees(double fees) { this.fees = fees; }
}
```

4. JDBC DAO – StudentDAO.java

java

 Copy code

```
package com.student.dao;

import java.sql.*;
import java.util.*;
import com.student.model.Student;

public class StudentDAO {
    static Connection con;

    public static Connection getConnection() {
        try {
            if (con == null) {
                Class.forName("com.mysql.cj.jdbc.Driver");
                con = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/studentdb", "root", "root"
                );
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return con;
    }

    // INSERT
    public static int addStudent(Student s) {
        int status = 0;
        try {
            PreparedStatement ps = getConnection().prepareStatement(
                "INSERT INTO students(name, email, course, fees) VALUES(?,?,?,?)"
            );
            ps.setString(1, s.getName());
            ps.setString(2, s.getEmail());
            ps.setString(3, s.getCourse());
            ps.setDouble(4, s.getFees());

            status = ps.executeUpdate();
        } catch (Exception e) { e.printStackTrace(); }
        return status;
    }
}
```

```
}

// LOAD ALL STUDENTS
public static List<Student> getAllStudents() {
    List<Student> list = new ArrayList<>();
    try {
        PreparedStatement ps = getConnection().prepareStatement("SELECT * FROM
        ResultSet rs = ps.executeQuery();

        while (rs.next()) {
            Student s = new Student();
            s.setId(rs.getInt("id"));
            s.setName(rs.getString("name"));
            s.setEmail(rs.getString("email"));
            s.setCourse(rs.getString("course"));
            s.setFees(rs.getDouble("fees"));
            list.add(s);
        }
    } catch (Exception e) { e.printStackTrace(); }
    return list;
}

// LOAD SINGLE STUDENT
public static Student getStudentById(int id) {
    Student s = null;
    try {
        PreparedStatement ps = getConnection().prepareStatement(
            "SELECT * FROM students WHERE id=?"
        );
        ps.setInt(1, id);
        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            s = new Student();
            s.setId(rs.getInt("id"));
            s.setName(rs.getString("name"));
            s.setEmail(rs.getString("email"));
            s.setCourse(rs.getString("course"));
            s.setFees(rs.getDouble("fees"));
        }
    }
}
```

```
    } catch (Exception e) { e.printStackTrace(); }
    return s;
}

// UPDATE
public static int updateStudent(Student s) {
    int status = 0;
    try {
        PreparedStatement ps = getConnection().prepareStatement(
            "UPDATE students SET name=?, email=?, course=?, fees=? WHERE id=?
        );
        ps.setString(1, s.getName());
        ps.setString(2, s.getEmail());
        ps.setString(3, s.getCourse());
        ps.setDouble(4, s.getFees());
        ps.setInt(5, s.getId());

        status = ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); }
    return status;
}

// DELETE
public static int deleteStudent(int id) {
    int status = 0;
    try {
        PreparedStatement ps = getConnection().prepareStatement(
            "DELETE FROM students WHERE id=?"
        );
        ps.setInt(1, id);
        status = ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); }
    return status;
}
}
```

5. Servlet Codes

+ AddStudentServlet.java

java

 Copy code

```
package com.student.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.student.dao.StudentDAO;
import com.student.model.Student;

public class AddStudentServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        String name = req.getParameter("name");
        String email = req.getParameter("email");
        String course = req.getParameter("course");
        double fees = Double.parseDouble(req.getParameter("fees"));

        Student s = new Student(name, email, course, fees);
        StudentDAO.addStudent(s);

        res.sendRedirect("list_students.jsp");
    }
}
```

EditStudentServlet.java

java

 Copy code

```
package com.student.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.student.dao.StudentDAO;
```

```
import com.student.model.Student;

public class EditStudentServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));
        Student s = StudentDAO.getStudentById(id);

        req.setAttribute("student", s);
        RequestDispatcher rd = req.getRequestDispatcher("edit_student.jsp");
        rd.forward(req, res);
    }
}
```

UpdateStudentServlet.java

java

 Copy code

```
package com.student.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.student.model.Student;
import com.student.dao.StudentDAO;

public class UpdateStudentServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));
        String name = req.getParameter("name");
        String email = req.getParameter("email");
        String course = req.getParameter("course");
        double fees = Double.parseDouble(req.getParameter("fees"));

        Student s = new Student(name, email, course, fees);
        s.setId(id);
    }
}
```

```
        StudentDAO.updateStudent(s);
        res.sendRedirect("list_students.jsp");
    }
}
```

✖ DeleteStudentServlet.java

java

 Copy code

```
package com.student.servlet;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import com.student.dao.StudentDAO;

public class DeleteStudentServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));
        StudentDAO.deleteStudent(id);

        res.sendRedirect("list_students.jsp");
    }
}
```

6. web.xml

xml

 Copy code

```
<web-app>

    <servlet>
        <servlet-name>addStudent</servlet-name>
        <servlet-class>com.student.servlet.AddStudentServlet</servlet-class>
```

```
</servlet>
<servlet-mapping>
  <servlet-name>addStudent</servlet-name>
  <url-pattern>/addStudent</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>editStudent</servlet-name>
  <servlet-class>com.student.servlet.EditStudentServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>editStudent</servlet-name>
  <url-pattern>/editStudent</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>updateStudent</servlet-name>
  <servlet-class>com.student.servlet.UpdateStudentServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>updateStudent</servlet-name>
  <url-pattern>/updateStudent</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>deleteStudent</servlet-name>
  <servlet-class>com.student.servlet.DeleteStudentServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>deleteStudent</servlet-name>
  <url-pattern>/deleteStudent</url-pattern>
</servlet-mapping>

</web-app>
```



7. JSP Pages

index.html

html

 Copy code

```
<h2>Student Management System</h2>
<a href="list_students.jsp">View Students</a><br>
<a href="add_student.jsp">Add Student</a>
```

add_student.jsp

jsp

 Copy code

```
<h2>Add Student</h2>

<form action="addStudent" method="post">
  Name: <input type="text" name="name"><br>
  Email: <input type="text" name="email"><br>
  Course: <input type="text" name="course"><br>
  Fees: <input type="number" name="fees"><br>
  <input type="submit" value="Add">
</form>
```

list_students.jsp

jsp

 Copy code

```
<%@ page import="java.util.*, com.student.dao.StudentDAO, com.student.model.Stude

<h2>All Students</h2>
<a href="add_student.jsp">Add New Student</a><br><br>

<table border="1" cellpadding="10">
<tr>
  <th>ID</th>
  <th>Name</th>
  <th>Email</th>
  <th>Course</th>
```

```

        <th>Fees</th>
        <th>Actions</th>
    </tr>

    <%
        List<Student> list = StudentDAO.getAllStudents();
        for(Student s : list) {
    %>

    <tr>
        <td><%= s.getId() %></td>
        <td><%= s.getName() %></td>
        <td><%= s.getEmail() %></td>
        <td><%= s.getCourse() %></td>
        <td><%= s.getFees() %></td>
        <td>
            <a href="editStudent?id=<%= s.getId() %>">Edit</a> |
            <a href="deleteStudent?id=<%= s.getId() %>">Delete</a>
        </td>
    </tr>

    <% } %>

</table>

```

edit_student.jsp

jsp

 Copy code

```

<%@ page import="com.student.model.Student" %>

<%
    Student s = (Student)request.getAttribute("student");
    %>

    <h2>Edit Student</h2>

    <form action="updateStudent" method="post">

```

```
<input type="hidden" name="id" value="<%= s.getId() %>">  
Name: <input type="text" name="name" value="<%= s.getName() %>"><br>  
Email: <input type="text" name="email" value="<%= s.getEmail() %>"><br>  
Course: <input type="text" name="course" value="<%= s.getCourse() %>"><br>  
Fees: <input type="number" name="fees" value="<%= s.getFees() %>"><br>  
<input type="submit" value="Update">  
</form>
```



PROJECT COMPLETED!

If you want next:

- 2 Login + Registration System (Servlet + JDBC)
- 3 Employee CRUD (Servlet)
- 4 Hospital/Library Management

Just send "Next 2", "Next 3", etc.



Next 2

Perfect! Here is **Step 2: Project Structure Setup** with complete clarity.



Step 2: Create Project Structure (Folders & Files)

We will create these folders and files inside your **Dynamic Web Project** → StudentCRUD .



Project Folder Structure (Full Layout)

pgsql

Copy code

```
StudentCRUD/
|
|— WebContent/
|   |— index.html
|   |— add-student.html
|   |— edit-student.html
|   |
|   |— WEB-INF/
|       |— web.xml
|       |— lib/
|
|— src/
    |— Student.java
    |— StudentDAO.java
    |— AddStudentServlet.java
    |— ViewStudentsServlet.java
    |— EditStudentServlet.java
    |— UpdateStudentServlet.java
    |— DeleteStudentServlet.java
```

Explanation of Each File

1 index.html

→ Home page with links: Add Student | View Students

2 add-student.html

→ Form to add a new student

3 edit-student.html

→ Form to edit already existing student

4 web.xml

→ Servlet mapping file

5 Student.java

→ Model class

6 StudentDAO.java

→ Handles DB operations (Insert, Update, Delete, Select)

7 AddStudentServlet.java

→ Handles form submission for adding student

8 ViewStudentsServlet.java

→ Displays all students in table form

9 EditStudentServlet.java

→ Loads data of single student in edit form

10 UpdateStudentServlet.java

→ Updates student data

1 1 DeleteStudentServlet.java

→ Deletes a student



This structure is READY for coding

If you say **Next**, I will provide:

✓ Step 3: Create Database & Table

- SQL commands
- DB connection details

Then after that we will write all the servlet codes step-by-step.

Tell me — Should I continue with Step 3 (Database Setup)?



Yes

Great! ✔ Let's continue.

✔ Step 3: Database Setup (MySQL)

We will create a database and a table for storing student records.

1 Create Database

Open **MySQL Command Prompt**, **phpMyAdmin**, or **MySQL Workbench** and run:

sql

 Copy code

```
CREATE DATABASE studentdb;
```

2 Use the Database

sql

 Copy code

```
USE studentdb;
```

3 Create Student Table

sql

 Copy code

```
CREATE TABLE students (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    course VARCHAR(100),  
    fee INT  
);
```

4 Add Some Sample Records (Optional)

sql

 Copy code

```
INSERT INTO students(name, email, course, fee)
VALUES
('Rahul Sharma', 'rahul@gmail.com', 'Java', 15000),
('Priya Verma', 'priya@gmail.com', 'Python', 12000);
```

5 JDBC Driver JAR File

Required file:

✓ mysql-connector-j.jar

Download it from MySQL official website OR use the version already in your system.

Add to Project:

Place this file inside:

swift

 Copy code

StudentCRUD/WebContent/WEB-INF/lib/

Without this, your program cannot connect to MySQL.

Database setup is complete!

If you're ready, say **Next**, and I will give you:

✓ Step 4: Create Student Model Class (Student.java)

This class will store student data.

   ...

Yes

Great! Let's move to the next step.

✓ Step 4: Create Student Model Class — Student.java

This file represents a **single student object**.

Create it inside:

CSS

 Copy code

src/Student.java

Student.java (Complete Code)

java

 Copy code

```
package com.student.model;

public class Student {
    private int id;
    private String name;
    private String email;
    private String course;
    private int fee;

    // Constructors
    public Student() {}

    public Student(int id, String name, String email, String course, int fee) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.course = course;
        this.fee = fee;
    }

    public Student(String name, String email, String course, int fee) {
        this.name = name;
        this.email = email;
        this.course = course;
        this.fee = fee;
    }
}
```

```
}

// Getters and setters
public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getCourse() {
    return course;
}

public void setCourse(String course) {
    this.course = course;
}

public int getFee() {
    return fee;
}

public void setFee(int fee) {
```

```
        this.fee = fee;
    }
}
```

Explanation

This class contains:

✓ Fields

- id
- name
- email
- course
- fee

✓ Constructors

- Empty constructor
- Constructor with ID
- Constructor without ID (used when adding new student)

✓ Getter/Setter methods

Used to access and update data.

Model class is ready!

If you're ready, say **Next**, and I will give you:

✓ Step 5: Create StudentDAO (Database Operations)

This file will contain **JDBC** code for:

- Add student
- View all students
- Get student by ID
- Update student
- Delete student

Shall I continue?



Yes

Great! Let's build the most important backend class.

✓ Step 5: Create StudentDAO (Database Operations)

This class will handle all JDBC operations:

- ✓ Add
- ✓ View
- ✓ Get by ID
- ✓ Update
- ✓ Delete

Create file:

swift

Copy code

src/com/student/dao/StudentDAO.java

Complete StudentDAO.java Code

java

Copy code

```
package com.student.dao;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

import com.student.model.Student;

public class StudentDAO {
```

```
private static Connection getConnection() {
    Connection con = null;

    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb",
            "root",
            "root"    // change password if needed
        );
    } catch (Exception e) {
        e.printStackTrace();
    }

    return con;
}

// 1. Add a new student
public static int addStudent(Student s) {
    int status = 0;
    try {
        Connection con = getConnection();
        String sql = "INSERT INTO students(name, email, course, fee) VALUES (
        PreparedStatement ps = con.prepareStatement(sql);

        ps.setString(1, s.getName());
        ps.setString(2, s.getEmail());
        ps.setString(3, s.getCourse());
        ps.setInt(4, s.getFee());

        status = ps.executeUpdate();

        con.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return status;
}
```

// 2. View all students

```
public static List<Student> getAllStudents() {
    List<Student> list = new ArrayList<>();
    try {
        Connection con = getConnection();
        String sql = "SELECT * FROM students";
        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery();

        while (rs.next()) {
            Student s = new Student(
                rs.getInt("id"),
                rs.getString("name"),
                rs.getString("email"),
                rs.getString("course"),
                rs.getInt("fee")
            );
            list.add(s);
        }

        con.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}
```

// 3. Get student by ID

```
public static Student getStudentById(int id) {
    Student s = null;
    try {
        Connection con = getConnection();
        String sql = "SELECT * FROM students WHERE id=?";
        PreparedStatement ps = con.prepareStatement(sql);
        ps.setInt(1, id);

        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            s = new Student(
                rs.getInt("id"),
```

```
        rs.getString("name"),
        rs.getString("email"),
        rs.getString("course"),
        rs.getInt("fee")

    );
}

    con.close();
} catch (Exception e) {
    e.printStackTrace();
}
return s;
}

// 4. Update student
public static int updateStudent(Student s) {
    int status = 0;
    try {
        Connection con = getConnection();
        String sql = "UPDATE students SET name=?, email=?, course=?, fee=? WH
        PreparedStatement ps = con.prepareStatement(sql);

        ps.setString(1, s.getName());
        ps.setString(2, s.getEmail());
        ps.setString(3, s.getCourse());
        ps.setInt(4, s.getFee());
        ps.setInt(5, s.getId());

        status = ps.executeUpdate();
        con.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
    return status;
}

// 5. Delete student
public static int deleteStudent(int id) {
    int status = 0;
```

```
try {  
    Connection con = getConnection();  
    String sql = "DELETE FROM students WHERE id=?";  
    PreparedStatement ps = con.prepareStatement(sql);  
    ps.setInt(1, id);  
  
    status = ps.executeUpdate();  
    con.close();  
  
} catch (Exception e) {  
    e.printStackTrace();  
}  
return status;  
}
```

What this class does?

- ✓ Connects to MySQL (getConnection())
 - ✓ Adds student (addStudent())
 - ✓ Views all (getAllStudents())
 - ✓ View single by ID (getStudentById())
 - ✓ Updates data (updateStudent())
 - ✓ Deletes student (deleteStudent())
-

Backend is ready!

If you're ready, say **Next**, and I will give:

Step 6: Create All Servlets

1. AddStudentServlet
2. ViewStudentsServlet

3. EditStudentServlet
4. UpdateStudentServlet
5. DeleteStudentServlet

All with complete working code.

Shall I continue?



Yes

Great! Let's start writing the Servlets.

✓ Step 6: Create All Servlets (Full Working Code)

We will make 5 Servlets:

1. AddStudentServlet
2. ViewStudentsServlet
3. EditStudentServlet
4. UpdateStudentServlet
5. DeleteStudentServlet

1 AddStudentServlet.java

File path:

swift

 Copy code

src/com/student/servlet/AddStudentServlet.java

java

 Copy code

```
package com.student.servlet;
```

```
import java.io.IOException;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;

import com.student.dao.StudentDAO;
import com.student.model.Student;

public class AddStudentServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String name = req.getParameter("name");
        String email = req.getParameter("email");
        String course = req.getParameter("course");
        int fee = Integer.parseInt(req.getParameter("fee"));

        Student s = new Student(name, email, course, fee);

        int status = StudentDAO.addStudent(s);

        if (status > 0) {
            resp.sendRedirect("view");
        } else {
            resp.getWriter().println("Failed to add student!");
        }
    }
}
```

**2**

ViewStudentsServlet.java

File path:

swift

Copy code

src/com/student/servlet/ViewStudentsServlet.java

java

Copy code

```
package com.student.servlet;
```

```
import java.io.IOException;
import java.util.List;
import javax.servlet.*;
import javax.servlet.http.*;

import com.student.dao.StudentDAO;
import com.student.model.Student;

public class ViewStudentsServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        List<Student> list = StudentDAO.getAllStudents();
        req.setAttribute("students", list);

        RequestDispatcher rd = req.getRequestDispatcher("view-students.jsp");
        rd.forward(req, resp);
    }
}
```



3

EditStudentServlet.java

Used when clicking **Edit** button.

File path:

swift

Copy code

```
src/com/student/servlet/EditStudentServlet.java
```

java

Copy code

```
package com.student.servlet;

import java.io.IOException;
import javax.servlet.*;
import javax.servlet.http.*;
```

```
import com.student.dao.StudentDAO;
import com.student.model.Student;

public class EditStudentServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));
        Student s = StudentDAO.getStudentById(id);

        req.setAttribute("student", s);
        RequestDispatcher rd = req.getRequestDispatcher("edit-student.jsp");
        rd.forward(req, resp);
    }
}
```

UpdateStudentServlet.java

Updates the final edited data.

File path:

swift

 Copy code

src/com/student/servlet/UpdateStudentServlet.java

java

 Copy code

```
package com.student.servlet;

import java.io.IOException;
import javax.servlet.*;
import javax.servlet.http.*;

import com.student.dao.StudentDAO;
import com.student.model.Student;

public class UpdateStudentServlet extends HttpServlet {
```

```
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {

    int id = Integer.parseInt(req.getParameter("id"));
    String name = req.getParameter("name");
    String email = req.getParameter("email");
    String course = req.getParameter("course");
    int fee = Integer.parseInt(req.getParameter("fee"));

    Student s = new Student(id, name, email, course, fee);

    int status = StudentDAO.updateStudent(s);

    if (status > 0) {
        resp.sendRedirect("view");
    } else {
        resp.getWriter().println("Update Failed!");
    }
}
```



5 DeleteStudentServlet.java

Removes a record.

File path:

swift

Copy code

src/com/student/servlet/DeleteStudentServlet.java

java

Copy code

```
package com.student.servlet;
```

```
import java.io.IOException;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
import com.student.dao.StudentDAO;

public class DeleteStudentServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));

        StudentDAO.deleteStudent(id);

        resp.sendRedirect("view");
    }
}
```



Servlets Completed Successfully!

Now we need to create:



Step 7: JSP & HTML Pages

- index.html
- add-student.html
- view-students.jsp
- edit-student.jsp

And also the `web.xml` mappings.

Say **Next**, and I will give you **all JSP & HTML pages** with proper Bootstrap UI.



Next

Great! Now let's build the frontend pages.

✓ Step 7: Create All JSP & HTML Pages

We will create the following:

1. index.html
2. add-student.html
3. view-students.jsp
4. edit-student.jsp
5. web.xml (Servlet mappings)

All with clean UI + working links.

1 index.html

👉 Home page with two options: Add Student | View Students

Create inside:

```
bash
```

 Copy code

```
WebContent/index.html
```

```
html
```

 Copy code

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Management</title>
</head>
<body>
  <h2>Student Management System</h2>

  <a href="add-student.html">Add Student</a><br><br>
  <a href="view">View Students</a>

</body>
</html>
```

2 add-student.html

 Form to add new student

Create file:

bash

 Copy code

WebContent/add-student.html

html

 Copy code

```
<!DOCTYPE html>
<html>
<head>
  <title>Add Student</title>
</head>
<body>

  <h2>Add New Student</h2>

  <form action="add" method="post">
    Name: <input type="text" name="name"><br><br>
    Email: <input type="email" name="email"><br><br>
    Course: <input type="text" name="course"><br><br>
    Fee: <input type="number" name="fee"><br><br>

    <input type="submit" value="Add Student">
  </form>

</body>
</html>
```

3 view-students.jsp

 Shows all students + Edit/Delete buttons

Create file:

bash

WebContent/view-students.jsp

jsp

 Copy code

```
<%@ page import="java.util.*, com.student.model.Student" %>

<!DOCTYPE html>
<html>
<head>
    <title>View Students</title>
</head>
<body>

<h2>All Students</h2>

<table border="1" cellpadding="10">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
        <th>Course</th>
        <th>Fee</th>
        <th>Actions</th>
    </tr>

    <%
        List<Student> list = (List<Student>) request.getAttribute("students");

        for(Student s : list){
    %>

    <tr>
        <td><%= s.getId() %></td>
        <td><%= s.getName() %></td>
        <td><%= s.getEmail() %></td>
        <td><%= s.getCourse() %></td>
        <td><%= s.getFee() %></td>

        <td>
```

```

        <a href="edit?id=<%= s.getId() %>">Edit</a> |
        <a href="delete?id=<%= s.getId() %>">Delete</a>
    </td>
</tr>

<% } %>

</table>

<br>
<a href="add-student.html">Add New Student</a>

</body>
</html>

```



edit-student.jsp



Form loads existing data & updates it.

Create file:

bash

Copy code

WebContent/edit-student.jsp

jsp

Copy code

```

<%@ page import="com.student.model.Student" %>

<!DOCTYPE html>
<html>
<head>
    <title>Edit Student</title>
</head>
<body>

<%
    Student s = (Student) request.getAttribute("student");
%>

```

```
<h2>Edit Student</h2>
```

```
<form action="update" method="post">
```

```
    <input type="hidden" name="id" value="<%= s.getId() %>">
```

```
    Name: <input type="text" name="name" value="<%= s.getName() %>"><br><br>
```

```
    Email: <input type="email" name="email" value="<%= s.getEmail() %>"><br><br>
```

```
    Course: <input type="text" name="course" value="<%= s.getCourse() %>"><br><br>
```

```
    Fee: <input type="number" name="fee" value="<%= s.getFee() %>"><br><br>
```

```
    <input type="submit" value="Update Student">
```

```
</form>
```

```
</body>
```

```
</html>
```

5 web.xml (Servlet Mapping)

Create file:

```
bash
```

 Copy code

```
WebContent/WEB-INF/web.xml
```

```
xml
```

 Copy code

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    version="3.1">
```

```
    <servlet>
```

```
        <servlet-name>AddStudentServlet</servlet-name>
```

```
        <servlet-class>com.student.servlet.AddStudentServlet</servlet-class>
```

```
    </servlet>
```

```
<servlet-mapping>
  <servlet-name>AddStudentServlet</servlet-name>
  <url-pattern>/add</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>ViewStudentsServlet</servlet-name>
  <servlet-class>com.student.servlet.ViewStudentsServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>ViewStudentsServlet</servlet-name>
  <url-pattern>/view</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>EditStudentServlet</servlet-name>
  <servlet-class>com.student.servlet.EditStudentServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>EditStudentServlet</servlet-name>
  <url-pattern>/edit</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>UpdateStudentServlet</servlet-name>
  <servlet-class>com.student.servlet.UpdateStudentServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>UpdateStudentServlet</servlet-name>
  <url-pattern>/update</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>DeleteStudentServlet</servlet-name>
  <servlet-class>com.student.servlet.DeleteStudentServlet</servlet-class>
</servlet>
```

```
<servlet-mapping>
  <servlet-name>DeleteStudentServlet</servlet-name>
  <url-pattern>/delete</url-pattern>
</servlet-mapping>

</web-app>
```



Your JSP + HTML Frontend is Completed!

Now your project is 100% ready to run.

If you want, I can also give:

- ✓ Bootstrap UI version
- ✓ WAR file creation guide
- ✓ Deployment on Tomcat guide
- ✓ Login + Session + Authentication
- ✓ MVC pattern version
- ✓ Filters + Listeners
- ✓ JDBC Connection Pooling

Tell me what you want next!



War file creation

Here is the **WAR File Creation** concept explained in the simplest and most practical way





WAR File in Java (Servlet/JSP Web Applications)

A WAR (Web Application Archive) file is used to package and deploy Java web applications on servers like Apache Tomcat, JBoss, GlassFish, etc.

A WAR file contains:

- HTML , CSS , JS files
- Servlet .class files
- JSP files
- web.xml
- Libraries (.jar files)
- Directory structure



Standard Structure of a WAR File

pgsql

Copy code

MyApp.war

```
|
|
|— index.html / index.jsp
|— login.jsp
|
|— WEB-INF
|   |— web.xml
|   |— classes/      → compiled .class files
|   |— lib/          → jar files
|   └— views/        → JSP files (optional)
|
|— META-INF
```



How to Create WAR File in Eclipse

Step 1: Project must be Dynamic Web Project

- ✓ Created via Eclipse
- ✓ Must contain WEB-INF folder

Step 2: Export WAR

Follow this path:

- ➔ Right-click Project → Export
- ➔ Web → WAR file
- ➔ Next
- ➔ Choose Destination folder
- ➔ Click **Finish**

Your WAR file is now created! 🎉

Deploy WAR on Apache Tomcat

Method 1: Drop WAR in “webapps” folder

1. Copy `MyApp.war`
2. Paste into Tomcat's:

```
bash
```

 Copy code

```
apache-tomcat/webapps/
```

3. Start Tomcat → It auto-extracts and deploys.

Access the app:

```
arduino
```

 Copy code

```
http://localhost:8080/MyApp/
```

Method 2: Tomcat Manager App

1. Start Tomcat
2. Visit:

```
bash
```

 Copy code

```
http://localhost:8080/manager/html
```

3. Login
 4. Upload WAR → Deploy
-

Why WAR File is Important?

- Portable → Deploy anywhere
 - Standard format for Java EE applications
 - Easy to deploy/update
 - Maintains clean directory structure
-

Example WAR Contents

pgsql

 Copy code

```
MyApp/
├─ index.jsp
├─ login.jsp
├─ styles.css
├─ WEB-INF/
│   └─ web.xml
│   └─ classes/
│       └─ com.demo.LoginServlet.class
│   └─ lib/
│       └─ mysql-connector.jar
└─ META-INF/
```

If you want, I can also provide:

- ✓ WAR creation screenshot guide
- ✓ Sample mini servlet project (Login + Dashboard)
- ✓ Complete **Servlet** → **JSP** → **JDBC** project with WAR file

Would you like that?



Yes

Great! I'll give you a **complete mini servlet project** with full folder structure + code + WAR creation steps.

This is the simplest and best beginner servlet project:

Mini Servlet Project: Login System (Servlet + JSP + JDBC)

You will get:

- ✓ Project folder structure
- ✓ All files (Servlet, JSP, web.xml)
- ✓ JDBC code
- ✓ WAR creation steps
- ✓ How to deploy on Tomcat

1. Project Structure (WAR compatible)

pgsql

 Copy code

```
LoginApp/
|
|— index.jsp
|— home.jsp
|
|— WEB-INF/
|   |— web.xml
|   |— classes/
|       |— LoginServlet.class (generated after compile)
```

```
|    └─ lib/
|
|    └─ mysql-connector.jar
```

2. index.jsp (Login Form)

jsp

 Copy code

```
<!DOCTYPE html>
<html>
<body>
    <h2>Login Form</h2>
    <form action="login" method="post">
        Username: <input type="text" name="username" /><br><br>
        Password: <input type="password" name="password" /><br><br>
        <input type="submit" value="Login" />
    </form>
</body>
</html>
```

3. LoginServlet.java

Create inside:

src/com.demo.LoginServlet.java

java

 Copy code

```
package com.demo;

import java.io.IOException;
import java.sql.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
```

```
String uname = req.getParameter("username");
String pass = req.getParameter("password");

boolean status = false;

try {
    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/test", "root", "password");

    PreparedStatement ps = con.prepareStatement(
        "SELECT * FROM users WHERE username=? AND password=?"
    );

    ps.setString(1, uname);
    ps.setString(2, pass);

    ResultSet rs = ps.executeQuery();
    status = rs.next();

} catch (Exception e) {
    e.printStackTrace();
}

if (status) {
    req.getRequestDispatcher("home.jsp").forward(req, resp);
} else {
    resp.getWriter().println("Invalid Login!");
}
}
```

4. home.jsp

jsp

 Copy code

```
<%@ page language="java" %>
<html>
```

```
<body>
  <h2>Welcome! Login Successful 🎉</h2>
</body>
</html>
```

5. web.xml (Servlet Mapping)

Location: WEB-INF/web.xml

xml

 Copy code

```
<web-app>
  <servlet>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class>com.demo.LoginServlet</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/login</url-pattern>
  </servlet-mapping>
</web-app>
```

6. Database Table

sql

 Copy code

```
CREATE DATABASE test;

USE test;

CREATE TABLE users(
  username VARCHAR(50),
  password VARCHAR(50)
);
```

```
INSERT INTO users VALUES ("admin", "123");
```



7. How to Create WAR File in Eclipse

- 1 Right-click the project
- 2 Click **Export**
- 3 Select **Web** → **WAR File**
- 4 Choose destination → **Finish**

You will get:

Copy code

```
LoginApp.war
```



8. Deploy WAR on Tomcat

- Copy LoginApp.war to:

```
bash
```

Copy code

```
apache-tomcat/webapps/
```

- Start Tomcat
- Auto deploy will extract the folder

Access your project:

```
arduino
```

Copy code

```
http://localhost:8080/LoginApp/
```

If you want, I can also give you:

- 🔥 Employee CRUD Servlet Project
- 🔥 Product Inventory System
- 🔥 Complete Student Management System
- 🔥 WAR file screenshots
- 🔥 Maven-based Servlet Project

Which one would you like next?



Yes

Great! Tell me **which servlet project you want next**, and I will provide the **full code + explanation + folder structure + WAR-ready project**.

Choose one from below 📌

📌 Choose Your Next Servlet Project

1 Employee CRUD System (Add, Update, Delete, List Employees)

Includes:

- ✓ Servlet
- ✓ JSP Pages
- ✓ JDBC
- ✓ MVC Pattern
- ✓ WAR structure

2 Product Inventory Management System

Features:

- ✓ Add product
- ✓ Update product

- ✓ Delete product
 - ✓ View all products
 - ✓ MySQL + JSP + Servlet
-

3 Student Management System

Features:

- ✓ Register Student
 - ✓ Edit
 - ✓ Delete
 - ✓ View all student details
-

4 Login + Registration Full System

Includes:

- ✓ Login
 - ✓ Registration
 - ✓ Password encryption
 - ✓ Session tracking
 - ✓ Logout
 - ✓ JDBC
-

5 File Upload + Download Servlet Project

Uses:

- ✓ MultipartConfig
 - ✓ File upload to server
 - ✓ File download from server
-

Which one do you want?

Reply with:

👉 1, 2, 3, 4, or 5



Great choice! 🎉

Here is the **Complete File Upload + Download Servlet Project** with full code, folder structure, JSP pages, web.xml, and how to generate the WAR file.

1. Project Structure (WAR-Ready)

kotlin

 Copy code

```
FileApp/
|
├─ upload.jsp
├─ download.jsp
|
├─ uploads/                ← uploaded files stored here
|
├─ WEB-INF/
|   └─ web.xml
|   └─ classes/
|       └─ (compiled .class files)
|       └─ lib/
|           └─ (commons-fileupload.jar, commons-io.jar)
```

Required Libraries (Add to WEB-INF/lib)

You need 2 jars:

- ✓ commons-fileupload-1.4.jar
- ✓ commons-io-2.11.0.jar

2. upload.jsp (File Upload Form)

jsp

 Copy code

```
<!DOCTYPE html>
<html>
<body>
    <h2>Upload File</h2>

    <form action="upload" method="post" enctype="multipart/form-data">
        Select File: <input type="file" name="file" />
        <br><br>
        <input type="submit" value="Upload">
    </form>

</body>
</html>
```

3. UploadServlet.java

Location: src/com/demo/UploadServlet.java

java

 Copy code

```
package com.demo;

import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;
import org.apache.commons.fileupload.*;
import org.apache.commons.fileupload.disk.*;
import org.apache.commons.fileupload.servlet.*;

@WebServlet("/upload")
@MultipartConfig
public class UploadServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
```

```

        throws ServletException, IOException {

    String uploadPath = getServletContext().getRealPath("") + File.separator +

    File uploadDir = new File(uploadPath);
    if (!uploadDir.exists()) uploadDir.mkdir();

    try {
        DiskFileItemFactory factory = new DiskFileItemFactory();
        ServletFileUpload upload = new ServletFileUpload(factory);

        java.util.List<FileItem> files = upload.parseRequest(req);

        for (FileItem item : files) {
            if (!item.isFormField()) {

                String fileName = new File(item.getName()).getName();
                String filePath = uploadPath + File.separator + fileName;

                File storeFile = new File(filePath);
                item.write(storeFile);

                resp.getWriter().println("File uploaded successfully: " + file
            }
        }

    } catch (Exception e) {
        resp.getWriter().println("Error: " + e.getMessage());
    }

}
}

```



4. download.jsp (List Files for Download)

jsp

Copy code

```

<%@ page import="java.io.*" %>
<html>
<body>
<h2>Download Files</h2>

<%
    String path = application.getRealPath("/") + "uploads";
    File folder = new File(path);
    File[] files = folder.listFiles();

    if (files != null) {
        for (File file : files) {
%>
            <a href="download?file=<%= file.getName() %>">
                <%= file.getName() %>
            </a><br>
        }
    }
%>

</body>
</html>

```

5. DownloadServlet.java

java

 Copy code

```

package com.demo;

import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;

@WebServlet("/download")
public class DownloadServlet extends HttpServlet {

```

```
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {

    String fileName = req.getParameter("file");

    String filePath = getServletContext().getRealPath("") + "uploads" + File.

    File file = new File(filePath);

    if (!file.exists()) {
        resp.getWriter().println("File not found!");
        return;
    }

    resp.setContentType("application/octet-stream");
    resp.setHeader("Content-Disposition", "attachment; filename=" + fileName)

    FileInputStream fis = new FileInputStream(file);
    OutputStream os = resp.getOutputStream();

    byte[] buffer = new byte[1024];
    int bytesRead;

    while ((bytesRead = fis.read(buffer)) != -1) {
        os.write(buffer, 0, bytesRead);
    }

    fis.close();
    os.close();
}
```



6. web.xml (Servlet Mappings)

Location: WEB-INF/web.xml

xml

 Copy code

```
<web-app>

    <servlet>
        <servlet-name>UploadServlet</servlet-name>
        <servlet-class>com.demo.UploadServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>UploadServlet</servlet-name>
        <url-pattern>/upload</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>DownloadServlet</servlet-name>
        <servlet-class>com.demo.DownloadServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>DownloadServlet</servlet-name>
        <url-pattern>/download</url-pattern>
    </servlet-mapping>

</web-app>
```

7. WAR File Creation Steps (Eclipse)

- 1 Right-click the project
- 2 Select **Export**
- 3 Choose **Web** → **WAR file**
- 4 Select destination
- 5 Click **Finish**

This generates:

 Copy code

FileApp.war



8. Deploy WAR on Tomcat

👉 Copy FileApp.war to:

```
bash
```

Copy code

```
apache-tomcat/webapps/
```

👉 Start Tomcat

👉 It auto-extracts the folder

Open upload page:

```
bash
```

Copy code

```
http://localhost:8080/FileApp/upload.jsp
```

Open download page:

```
bash
```

Copy code

```
http://localhost:8080/FileApp/download.jsp
```



Project Completed!

If you want, I can also give you:

- 🔥 Zip file of full code
- 🔥 Project with MVC (Controller-Service-DAO)
- 🔥 Database version (store files in DB)
- 🔥 Advanced File Upload (multiple files, size limit, file type validation)

Just tell me Yes.



Advanced file upload

Great! Here is the **Advanced File Upload Servlet Project** — with multiple file upload, file type validation, size restriction, progress support, and secure storage.

You will get **full folder structure + JSP + Servlets + web.xml + advanced validations**.

ADVANCED FILE UPLOAD PROJECT (Servlet + JSP + Apache FileUpload + Validation)

- ✓ Upload multiple files
 - ✓ Restrict file type (Only images/PDF allowed)
 - ✓ Max file size limit
 - ✓ Store files in a secure `/uploads` folder
 - ✓ Show uploaded files with download links
 - ✓ WAR-ready project
-

1. Project Folder Structure

pgsql

 Copy code

```
AdvancedFileApp/
|
├─ upload.jsp
├─ upload-success.jsp
├─ list-files.jsp
|
├─ uploads/          ← secure storage
|
├─ WEB-INF/
|   └─ web.xml
|   └─ classes/
|   └─ lib/
```

```
| commons-fileupload-1.4.jar  
| commons-io-2.11.0.jar
```

2. upload.jsp (Advanced Multi-file Upload Form)

jsp

 Copy code

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>Advanced File Upload</h2>  
  
<form action="upload" method="post" enctype="multipart/form-data">  
  Select Files: <input type="file" name="file" multiple>  
  <br><br>  
  <input type="submit" value="Upload Files">  
</form>  
  
</body>  
</html>
```

3. UploadServlet (Multiple Upload + Validation)

java

 Copy code

```
package com.demo;  
  
import java.io.*;  
import java.util.List;  
import jakarta.servlet.*;  
import jakarta.servlet.http.*;  
import jakarta.servlet.annotation.*;  
import org.apache.commons.fileupload.*;  
import org.apache.commons.fileupload.disk.*;  
import org.apache.commons.fileupload.servlet.*;  
import org.apache.commons.io.FilenameUtils;
```

```
@WebServlet("/upload")
public class UploadServlet extends HttpServlet {

    // Allowed file types
    private static final String[] ALLOWED_TYPES = {"jpg", "jpeg", "png", "pdf"};

    // Max file size: 5 MB
    private static final long MAX_FILE_SIZE = 5 * 1024 * 1024;

    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String uploadPath = getServletContext().getRealPath("") + "uploads";
        File uploadDir = new File(uploadPath);
        if (!uploadDir.exists()) uploadDir.mkdir();

        DiskFileItemFactory factory = new DiskFileItemFactory();
        factory.setSizeThreshold(1024 * 1024);

        ServletFileUpload upload = new ServletFileUpload(factory);
        upload.setFileSizeMax(MAX_FILE_SIZE);
        upload.setSizeMax(10 * MAX_FILE_SIZE);

        try {
            List<FileItem> formItems = upload.parseRequest(req);

            for (FileItem item : formItems) {

                if (!item.isFormField()) {

                    String fileName = FilenameUtils.getName(item.getName());
                    String ext = FilenameUtils.getExtension(fileName).toLowerCase();

                    // Validate file type
                    boolean validType = false;
                    for (String type : ALLOWED_TYPES) {
                        if (ext.equals(type)) validType = true;
                    }
                }
            }
        }
    }
}
```

```

        if (!validType) {
            resp.getWriter().println("Invalid file type: " + fileName);
            continue;
        }

        // Validate file size
        if (item.getSize() > MAX_FILE_SIZE) {
            resp.getWriter().println("File too large: " + fileName);
            continue;
        }

        // Store file
        File storeFile = new File(uploadPath + File.separator + fileName);
        item.write(storeFile);

        resp.getWriter().println("Uploaded: " + fileName + "<br>");
    }
}

} catch (Exception ex) {
    resp.getWriter().println("Error: " + ex.getMessage());
}
}
}

```

4. list-files.jsp (List Uploaded Files)

jsp

 Copy code

```

<%@ page import="java.io.*" %>
<html>
<body>

<h2>Uploaded Files</h2>

<%
    String path = application.getRealPath("/") + "uploads";
    File folder = new File(path);

```

```

        File[] files = folder.listFiles();

        if(files != null){
            for(File f : files){
%>
                <a href="download?file=<%= f.getName() %>">
                    <%= f.getName() %>
                </a><br>
            <%
                }
            }
        %>

    </body>
</html>

```

5. DownloadServlet

java

 Copy code

```

package com.demo;

import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;

@WebServlet("/download")
public class DownloadServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String fileName = req.getParameter("file");
        String filePath = getServletContext().getRealPath("/") + "uploads/" + fileName;

        File file = new File(filePath);

        if (!file.exists()) {
            resp.getWriter().println("File not found!");
        }
    }
}

```

```
        return;
    }

    resp.setContentType("application/octet-stream");
    resp.setHeader("Content-Disposition", "attachment;filename=" + fileName);

    FileInputStream fis = new FileInputStream(file);
    OutputStream os = resp.getOutputStream();

    byte[] buffer = new byte[4096];
    int bytesRead;

    while ((bytesRead = fis.read(buffer)) != -1) {
        os.write(buffer, 0, bytesRead);
    }

    fis.close();
    os.close();
}
}
```



6. web.xml (Advanced Mapping)

xml

 Copy code

```
<web-app>

    <servlet>
        <servlet-name>UploadServlet</servlet-name>
        <servlet-class>com.demo.UploadServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <url-pattern>/upload</url-pattern>
        <servlet-name>UploadServlet</servlet-name>
    </servlet-mapping>

    <servlet>
        <servlet-name>DownloadServlet</servlet-name>
```

```
<servlet-class>com.demo.DownloadServlet</servlet-class>
</servlet>
<servlet-mapping>
  <url-pattern>/download</url-pattern>
  <servlet-name>DownloadServlet</servlet-name>
</servlet-mapping>

</web-app>
```

7. Extra Security Features (Optional But Recommended)

I can also add:

✓ File renaming (UUID prefix)

Prevents overwriting:

 Copy code

```
uuid_randomname.ext
```

✓ Store files *outside webapps*

Cannot be accessed directly.

✓ Database store: filename, size, type, upload time

✓ Upload progress bar (AJAX + Servlet)

If you want, I can provide:

- ✓ ZIP of entire project
- ✓ Maven version of same project
- ✓ Full UI (Bootstrap)
- ✓ Validation with JavaScript
- ✓ File upload preview

- ✓ Excel/CSV upload handling
- ✓ Uploading video files with size > 1GB

Would you like **Advanced File Upload + Progress Bar + File Rename + Security** version also?

Reply **Yes**.



Yes

Thought for 6s >

Nice — here's a ready-to-drop-in **Advanced File Upload** implementation that includes:

- Multiple-file upload
- Client-side **progress bar** (AJAX/XHR)
- File-type & size validation (client + server)
- **UUID renaming** to prevent collisions
- Storage **outside the webapp** (safer) and configurable via `context-param`
- Safe filename handling (sanitization)
- Download & list pages
- `@MultipartConfig` (no extra jars required) — works with Servlet 3.0+ (Tomcat 8+)

Copy the files into your Dynamic Web Project, change the upload directory path once, build WAR and deploy.

1) Project structure (WAR-ready)

pgsql

Copy code

AdvancedSecureUpload/

|

├─ upload.jsp

<-- upload page with AJAX progress

├─ list-files.jsp

<-- list uploaded files (download links)

```

├─ WEB-INF/
│   │   └─ web.xml
│   │   └─ classes/
│   │       └─ (compiled servlets)
└─ (uploads/)          <-- recommended: put outside this folder on server

```

We will store files **outside** the webapp (e.g. /var/uploads/AdvancedSecureUpload or C:\uploads\AdvancedSecureUpload). You will set that path in web.xml (context-param) or in Tomcat context.

2) upload.jsp — AJAX uploader with progress bar

Create WebContent/upload.jsp (Or upload.html):

html

 Copy code

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Advanced File Upload (with Progress)</title>
  <style>
    #progressContainer { width: 100%; max-width: 600px; margin: 12px 0; }
    .progressBar { width: 0%; height: 18px; background:#4caf50; }
    .progressWrap { width:100%; background:#ddd; }
    .fileRow { margin:8px 0; }
  </style>
</head>
<body>
  <h2>Advanced File Upload</h2>

  <input id="fileInput" type="file" multiple />
  <br><br>

  <!-- restrictions displayed to user -->
  <div>
    Allowed types: <strong>jpg, jpeg, png, pdf</strong> – Max per-file: <strong>5
  </div>

```

```
<div id="fileList"></div>

<button id="uploadBtn">Upload</button>

<div id="progressContainer"></div>

<script>
const ALLOWED = ['jpg', 'jpeg', 'png', 'pdf'];
const MAX_SIZE = 5 * 1024 * 1024; // 5MB

document.getElementById('uploadBtn').addEventListener('click', () => {
  const input = document.getElementById('fileInput');
  const files = input.files;
  if (!files || files.length === 0) { alert('Select files first'); return; }

  // Clear UI
  document.getElementById('fileList').innerHTML = '';
  document.getElementById('progressContainer').innerHTML = '';

  // Build FormData
  const fd = new FormData();
  for (let i=0; i<files.length; i++) {
    const f = files[i];
    const ext = f.name.split('.').pop().toLowerCase();
    if (!ALLOWED.includes(ext)) {
      alert('Skipped (invalid type): ' + f.name);
      continue;
    }
    if (f.size > MAX_SIZE) {
      alert('Skipped (too large): ' + f.name);
      continue;
    }
    fd.append('files', f);
    // create progress UI per file
    const row = document.createElement('div'); row.className = 'fileRow';
    row.innerHTML = `<strong>${f.name}</strong> (${Math.round(f.size/1024)} KB)
      <div class="progressWrap"><div class="progressBar" id="bar-${i}"></div></div>
      <div id="status-${i}"></div>`;
    document.getElementById('fileList').appendChild(row);
  }
}
```

```
}

// Send via XHR (we'll track overall progress and per-file approximations)
const xhr = new XMLHttpRequest();
xhr.open('POST', 'upload', true);

// overall progress (bytes)
xhr.upload.onprogress = function(e) {
  if (e.lengthComputable) {
    const percent = Math.round((e.loaded / e.total) * 100);
    // set all bars to percent as approximation (server-side chunking may differ)
    const bars = document.querySelectorAll('.progressBar');
    bars.forEach(b => b.style.width = percent + '%');
  }
};

xhr.onreadystatechange = function() {
  if (xhr.readyState === 4) {
    if (xhr.status === 200) {
      // server returns JSON lines per file
      try {
        const resp = JSON.parse(xhr.responseText);
        // resp is array of {originalName, storedName, status, message}
        resp.forEach((r, idx) => {
          const st = document.getElementById('status-' + idx);
          if (!st) return;
          if (r.status === 'ok') {
            st.innerHTML = `Uploaded → ${r.storedName}`;
          } else {
            st.innerHTML = `Error: ${r.message}`;
            document.getElementById('bar-' + idx).style.background = '#f44336';
          }
        });
      } catch (ex) {
        alert('Upload finished but response parsing failed: ' + ex);
      }
    } else {
      alert('Upload failed. Server status: ' + xhr.status);
    }
  }
}
```

```

    };

    xhr.send(fd);
  });
</script>
</body>
</html>

```

Notes:

- Client validates file types & sizes before sending to reduce round-trips.
- Progress shown per-file as an approximation using overall upload progress (fine-grained per-file progress would require separate requests per file).

3) UploadServlet (server-side) — secure, multi-file, rename with UUID, validation

Create `src/com/advanced/UploadServlet.java` (package path as you prefer). This uses standard Servlet API (`javax.servlet`) and `@MultipartConfig` — no external libraries required.

java

 Copy code

```

package com.advanced;

import java.io.*;
import java.nio.file.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.annotation.MultipartConfig;
import javax.servlet.http.*;
import org.json.JSONArray;
import org.json.JSONObject; // optional -> if you don't have org.json remove and i

@MultipartConfig(
    fileSizeThreshold = 1024 * 1024, // 1MB memory threshold
    maxFileSize = 5 * 1024 * 1024,   // 5MB per file
    maxRequestSize = 50 * 1024 * 1024 // 50MB overall

```

```
)  
public class UploadServlet extends HttpServlet {  
  
    // Allowed extensions  
    private static final Set<String> ALLOWED = new HashSet<>(  
        Arrays.asList("jpg", "jpeg", "png", "pdf")  
    );  
  
    private Path uploadDir;  
  
    @Override  
    public void init() throws ServletException {  
        // read upload.dir from context-param; fallback to app's 'uploads' folder  
        String configured = getServletContext().getInitParameter("upload.dir");  
        if (configured != null && !configured.trim().isEmpty()) {  
            uploadDir = Paths.get(configured);  
        } else {  
            uploadDir = Paths.get(getServletContext().getRealPath("/"), "uploads")  
        }  
  
        try {  
            if (!Files.exists(uploadDir)) {  
                Files.createDirectories(uploadDir);  
            }  
        } catch (IOException e) {  
            throw new ServletException("Could not create upload dir: " + uploadDir, e);  
        }  
    }  
  
    @Override  
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)  
        throws ServletException, IOException {  
  
        // We'll collect results and return JSON array  
        List<Map<String, Object>> results = new ArrayList<>();  
  
        Collection<Part> parts = req.getParts(); // requires form field name 'file'  
        for (Part part : parts) {  
            // Skip form fields  
            if (part.getSubmittedFileName() == null) continue;  
        }  
    }  
}
```

```
String originalName = Paths.get(part.getSubmittedFileName()).getFileName();
String ext = getExtension(originalName).toLowerCase();

Map<String, Object> fileResult = new HashMap<>();
fileResult.put("originalName", originalName);

// Validate extension
if (!ALLOWED.contains(ext)) {
    fileResult.put("status", "error");
    fileResult.put("message", "Invalid file type: " + ext);
    results.add(fileResult);
    // skip saving
    continue;
}

// Validate size (server-side)
long size = part.getSize();
if (size == 0) {
    fileResult.put("status", "error");
    fileResult.put("message", "Empty file");
    results.add(fileResult);
    continue;
}

// file size limit is enforced by @MultipartConfig; still we check
long maxFile = 5L * 1024 * 1024;
if (size > maxFile) {
    fileResult.put("status", "error");
    fileResult.put("message", "File too large");
    results.add(fileResult);
    continue;
}

// Generate unique name: uuid_original.ext (or just uuid.ext)
String uuid = UUID.randomUUID().toString();
String storedName = uuid + "_" + originalName; // preserve original name

Path out = uploadDir.resolve(storedName);
```

```
// Save file (stream copy)
try (InputStream in = part.getInputStream()) {
    // Use atomic move pattern: write to temp then move
    Path tmp = Files.createTempFile(uploadDir, "upload_", ".tmp");
    try (OutputStream os = Files.newOutputStream(tmp, StandardOpenOpt
        byte[] buffer = new byte[8192];
        int len;
        while ((len = in.read(buffer)) != -1) {
            os.write(buffer, 0, len);
        }
    }

    Files.move(tmp, out, StandardCopyOption.ATOMIC_MOVE, StandardCopy
    // Set secure permissions (optional, platform-dependent)
    try {
        out.toFile().setReadable(true, true);
        out.toFile().setWritable(true, true);
    } catch (Exception ignore) {}

    fileResult.put("status", "ok");
    fileResult.put("storedName", storedName);
    fileResult.put("size", size);
} catch (Exception ex) {
    fileResult.put("status", "error");
    fileResult.put("message", "Save failed: " + ex.getMessage());
}

results.add(fileResult);
}

// Return JSON array
resp.setContentType("application/json");
// If org.json not available, build JSON manually
String json = buildJson(results);
resp.getWriter().write(json);
}

private String getExtension(String name) {
    int i = name.lastIndexOf('.');
    if (i >= 0 && i < name.length()-1) return name.substring(i+1);
}
```

```

        return "";
    }

    private String buildJson(List<Map<String,Object>> results) {
        // minimal manual JSON builder
        StringBuilder sb = new StringBuilder();
        sb.append('[');
        boolean first = true;
        for (Map<String,Object> r : results) {
            if (!first) sb.append(',');
            sb.append('{');
            boolean innerFirst = true;
            for (Map.Entry<String,Object> e : r.entrySet()) {
                if (!innerFirst) sb.append(',');
                innerFirst = false;
                sb.append('').append(escapeJson(e.getKey())).append('').append(
                    Object v = e.getValue();
                    if (v instanceof Number) sb.append(v);
                    else sb.append('').append(escapeJson(String.valueOf(v))).append(
                }
                sb.append('}');
                first = false;
            }
            sb.append('');
        }
        return sb.toString();
    }

    private String escapeJson(String s) {
        return s.replace("\\", "\\\\").replace("\"", "\\\"").replace("\n", "\\n").re
    }
}

```

Notes:

- `@MultipartConfig` enforces max file size. The servlet also validates file type and size.
- Upload folder is taken from `context-param upload.dir` in `web.xml` (see next). If not provided, it creates `uploads/` inside the webapp (but recommended to configure external folder).
- Files are saved with `UUID_originalName` to avoid collision while retaining the original name.

4) list-files.jsp — show files with download links

Create WebContent/list-files.jsp :

jsp

 Copy code

```
<%@ page import="java.nio.file.*, java.util.*" %>
<%
    String configured = application.getInitParameter("upload.dir");
    String uploadsPath;
    if (configured != null && !configured.isEmpty()) {
        uploadsPath = configured;
    } else {
        uploadsPath = application.getRealPath("/") + "uploads";
    }

    java.nio.file.Path dir = Paths.get(uploadsPath);
    List<String> files = new ArrayList<>();
    if (Files.exists(dir) && Files.isDirectory(dir)) {
        try (DirectoryStream<Path> ds = Files.newDirectoryStream(dir)) {
            for (Path p : ds) {
                files.add(p.getFileName().toString());
            }
        } catch (Exception e) { out.println("Error listing files: " + e); }
    }
%>

<html>
<body>
<h2>Uploaded Files</h2>
<ul>
<% for (String f : files) { %>
    <li><a href="download?file=<%= java.net.URLEncoder.encode(f, "UTF-8") %>">
        <%= f %></a></li>
<% } %>
</ul>
</body>
</html>
```

5) DownloadServlet — stream files securely

Create `src/com/advanced/DownloadServlet.java` :

java

 Copy code

```
package com.advanced;

import java.io.*;
import java.net.URLEncoder;
import java.nio.file.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DownloadServlet extends HttpServlet {

    private Path uploadDir;

    @Override
    public void init() throws ServletException {
        String configured = getServletContext().getInitParameter("upload.dir");
        if (configured != null && !configured.trim().isEmpty()) {
            uploadDir = Paths.get(configured);
        } else {
            uploadDir = Paths.get(getServletContext().getRealPath("/"), "uploads")
        }
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String fileParam = req.getParameter("file");
        if (fileParam == null || fileParam.trim().isEmpty()) {
            resp.getWriter().println("Invalid file parameter");
            return;
        }

        // Avoid path traversal by only allowing file name, not path
        String fileName = Paths.get(fileParam).getFileName().toString();
```

```

Path filePath = uploadDir.resolve(fileName);
if (!Files.exists(filePath) || !Files.isRegularFile(filePath)) {
    resp.setStatus(HttpServletResponse.SC_NOT_FOUND);
    resp.getWriter().println("File not found");
    return;
}

// set content type (basic)
resp.setContentType(getServletContext().getMimeType(fileName) == null
    ? "application/octet-stream"
    : getServletContext().getMimeType(fileName));

// Suggest original filename (part after first underscore if we used uuid_
String suggested = fileName;
int idx = fileName.indexOf('_');
if (idx != -1 && idx < fileName.length()-1) {
    suggested = fileName.substring(idx+1);
}

String header = "attachment; filename=\"" + URLEncoder.encode(suggested,
resp.setHeader("Content-Disposition", header);

try (InputStream in = Files.newInputStream(filePath);
    OutputStream out = resp.getOutputStream()) {

    byte[] buffer = new byte[8192];
    int len;
    while ((len = in.read(buffer)) != -1) {
        out.write(buffer, 0, len);
    }
}
}
}

```

6) web.xml — context-param and servlet mappings

Create WebContent/WEB-INF/web.xml :

xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    version="3.1">

    <!-- Configure external upload directory here -->
    <context-param>
        <param-name>upload.dir</param-name>
        <!-- Example Linux: /var/uploads/AdvancedSecureUpload
            Example Windows: C:\uploads\AdvancedSecureUpload -->
        <param-value>/var/uploads/AdvancedSecureUpload</param-value>
    </context-param>

    <servlet>
        <servlet-name>UploadServlet</servlet-name>
        <servlet-class>com.advanced.UploadServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>UploadServlet</servlet-name>
        <url-pattern>/upload</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>DownloadServlet</servlet-name>
        <servlet-class>com.advanced.DownloadServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>DownloadServlet</servlet-name>
        <url-pattern>/download</url-pattern>
    </servlet-mapping>

</web-app>
```

Important: Replace `/var/uploads/AdvancedSecureUpload` with your actual folder path. Make sure Tomcat user has read/write permissions on that folder.